Geometry- and Physics-Guided IR Train Proof Contract

Immersive Railroading 1.11.0 / Minecraft 1.7.10

2026-08-16

NOT_READY. This is a proof and implementation-contract artifact. The production Lua controller is unchanged. All previous PID proof, optimality, and readiness claims are withdrawn.

Contents

1 Authority, scope, and classifications **1**

2 Provenance and Java-8 source boundary **2**

3 Trace-centre and spline model **2**
3.1 All reconstruction variables . . . 2
3.2 Incremental reconstruction and topology . . . 3

4 Routing graph, stations, signals, and active routes **3**

5 Detector profile and source-separated data **3**

6 Directed front-coupler coordinate **4**

7 IR physics derivation **4**
7.1 Source records and constants . . . 4
7.2 Line-by-line force ledger . . . 4
7.3 Particle aggregation . . . 5

8 Combined command and plant transition **5**

9 Uniform guard, safety, arrival, and holding **5**

10 Producer/consumer theorem inventory **6**
10.1 Required spline-to-edge proof . . . 6

11 Anti-cheating audit and gate result **6**

12 Implementation contract and readiness **6**

1 Authority, scope, and classifications

The authoritative specifications are `PLAN_NEWPROOF.md` and its overriding supplement `PLAN_NEWPROOF_REVISION.md`. The proof target is a robust stopping and terminal-arrival contract for a directed station or signal. It is not a PID proof and it does not claim controller optimality.

The source-connected funnel is:

$$\text{tracer samples} \to \text{track-centre splines} \to \text{routing graph} \to$$
$$\text{detector profile} \to \text{front coupler} \to \text{combined command} \to$$
$$\text{IR plant} \to \text{arrival/holding}.$$

The spline is geometric source of truth. The graph is only a routing abstraction and never fits or rewrites a spline. Every statement is labelled as one of: proved by Lean; bytecode fact; derived from bytecode; certified model assumption; runtime contract; future Lua implementation obligation; or unsupported/withdrawn.

2 Provenance and Java-8 source boundary

The extracted directory `immersive_railroading/.cache/jar/` is authoritative. Its manifest identifies Minecraft 1.7.10, Immersive Railroading 1.11.0, and `Multi-Release: true`. Root class files have major version 52 (Java 8); `META-INF/versions/16` classes have major version 60 and are excluded. Root class and complete decompiler-input hashes are recorded in `pid-jar-manifest.md`; the six-input reproducibility check is recorded in `pid-physics-contract-manifest.sha256`. The exact branch ledger is in `pid-bytecode-traceability.md`.

The root `javap -p -c` evidence establishes the following boundaries:

- `CommonAPI.info()` provides dynamic identity, definition, weight, speed, direction, brake/control, and locomotive fields; `consist()` is aggregate validation only.
- The detector manager emits `ir_train_overhead` with a stock UUID, and detector `info()` delegates through the API.
- `EntityRollingStock` position is not proved to be the F3 hitbox or rail centreline. `SimulationState.Configuration` and `calculateCouplerPositions()` consume bogey, coupler, yaw, track, mass, slack, and resistance data.
- `CubicCurve.position()` is a cubic Bernstein expression; `lengthWithCache()` samples derivatives and accumulates trapezoids.
- `SimulationState.forcesNewtons()` computes signed grade plus tractive effort. `frictionNewtons()` computes the resistance and brake branches listed in Section 7.
- `SimulationState.next()` clones state, calls movement, has an unchanged-position velocity-zero branch, recomputes couplers/collisions and direct resistance. `Consist$Particle` updates force, friction, linkage, position, and velocity per particle.

3 Trace-centre and spline model

3.1 All reconstruction variables

The sample index is $i \in \mathbb{N}$. Each accepted tracer observation is

$$o_i = (t_i, r_i, u_i, h_i, \mathrm{uuid}_i, \mathrm{def}_i, b_i, \alpha_i, \Delta\ell_i, \kappa_i, \epsilon_{float,i}, \epsilon_{missing,i}, e_i),$$

where t_i is the tick, $r_i \in \mathbb{R}^3$ is raw `getPos()`, u_i is speed, h_i is heading/orientation, b_i is the compact tracer bogey/reference offset, $\alpha_i \geq 0$ is observation age, $\Delta\ell_i \geq 0$ is sampling spacing, $\kappa_i \geq 0$ is a curvature bound, $\epsilon_{float,i} \geq 0$ is numeric error, $\epsilon_{missing,i} \geq 0$ is missing/delayed-observation error, and $e_i \in \mathbb{R}^3$ is the measured residual. The centre estimate and bound are

$$c_i = r_i + b_i, \qquad \epsilon_{trace,i} = \Delta\ell_i\kappa_i + epsilon_{float,i} + \epsilon_{missing,i}, qquad c_i^{actual} = c_i + e_i, \quad \|e_i\|_1 \leq \epsilon_{trace,i}.$$

The front coupler is absent from this record. It is a stopping reference only. The offset, validity, and residual are a certified model assumption and runtime contract boundary until the future sampler supplies them; raw position is never silently reused.

For stable spline ID j, define four controls $P_{j,0}, P_{j,1}, P_{j,2}, P_{j,3} \in \mathbb{R}^3$ and

$$\sigma_j(t) = (1-t)^3 P_{j,0} + 3(1-t)^2 t P_{j,1} + 3(1-t)t^2 P_{j,2} + t^3 P_{j,3}, \quad 0 \leq t \leq 1.$$

The stored variables are

$$S_j = (j, P_{j,0:3}, A_j, L_j, \tau_j, g_j, \kappa_j, \epsilon_{fit,j}, T_j, M_j),$$

where A_j is a monotone arc-length table, $L_j > 0$ is total length, τ_j is tangent, g_j is grade, κ_j is curvature or a bound, $\epsilon_{fit,j} \geq 0$ is fitting error, T_j is the list of source trace IDs, and M_j is optional builder metadata. Straight track is a degenerate cubic. IR turns are not called exact circles: the root turn implementation uses a cubic approximation.

For each observation, the fit stores a parameter $q_i \in [0,1]$ and requires

$$\|c_i - \sigma_j(q_i)\|_1 \leq \epsilon_{fit,j} + \epsilon_{trace,i}.$$

The producer `validatedSplineProducer` returns a validated spline only when IDs, length, fit error, source traces, all observations, parameter rows, and residual rows pass. Thus the theorem does not accept an arbitrary caller certificate as proof of its own validity.

3.2 Incremental reconstruction and topology

For a new trace and existing spline, the match record is

$$m = (d_3, d_\tau, d_\kappa, d_s; \delta_3, \delta_\tau, \delta_\kappa, \delta_s, h, \delta_h),$$

where the first four values are 3-D position, tangent, curvature, and arc-coordinate discrepancies, and the deltas are certified thresholds. A match requires all four 3-D/arc inequalities; 2-D projection intersection is never enough. The topology class is one of continuation, extension, split, overlap, shared connection, isolated crossing, overpass, new geometry, or unresolved. A same-level pair whose paths both continue without connection is an isolated crossing. A height separation greater than tolerance is an overpass. Both preserve separate paths and are not routable. Unresolved branches remain unavailable; manual classification is an explicit later transition, never a silent guess.

The implementable state transition is:

1. validate observations and construct c_i and its residual terms;
2. fit and validate the cubic, arc marks, reverse controls, length, tangent, grade, curvature, and fit error;
3. compare against existing splines using all 3-D/arc metrics;
4. classify the topology, preserving unresolved/crossing/overpass state;
5. commit only validated spline/source metadata and revision;
6. rebuild the graph from validated spline IDs and explicit connections.

This is the certified model assumption and future Lua implementation obligation for the future persistent mapper.

4 Routing graph, stations, signals, and active routes

At graph revision r_G, define $G_{r_G} = (V_{r_G}, E_{r_G})$. A vertex is $v = (id, j, s)$ with $0 \le s \le L_j$. A directed edge is

$$e = (id_e, j, s_0, s_1, o, v_a, v_b), \qquad o \in \{+1, -1\},$$

referencing a validated spline ID and arc interval. A physical spline normally produces symmetric forward and reverse edges. A connection record is explicit evidence between validated endpoints; it is not inferred from a projected crossing.

A marker is

$$m = (id_m, p_m, \delta_m, a_m, k_m),$$

where p_m is its world point, δ_m is mapping tolerance, a_m is exactly one permitted approach orientation, and k_m is station or signal kind. The closest spline candidate is accepted only when distance is within δ_m and $0 \le s_m \le L_j$. The opposite approach requires a separate marker configuration.

An active route is $R = (r_G, S_R, V_R, E_R, O_R)$. It validates only its referenced splines, vertices, edges, intervals, endpoint sequence, orientation, and the current graph revision. It does not scan the whole graph. The graph-network producer consumes candidate traces, invokes the validated spline producer for each candidate, and rejects empty or duplicate stable identifiers rather than silently aliasing geometry. A changed active section invalidates R, invokes route finding against the latest graph, and fails closed with a report if no route exists. The Lean producers are `directedMarkerProducer`, `routingGraphProducer`, `graphNetworkProducer`, `routeReferencesGraph`, and `routeDecision`.

5 Detector profile and source-separated data

For each `ir_train_overhead` event, record u_k =`stock_uuid` and immediately call detector `info()`. The ordered record is

$$q_k = (u_k, d_k, w_k, v_k, a_k, b_k^{train}, b_k^{ind}, T_k, C_k, \eta_k^{sys}, \eta_k^{adh}, \mu_k^{shoe}),$$

containing UUID, definition ID, weight, direction, speed, both brake channels, traction T, cogging C, brake-system and adhesion efficiencies, and shoe friction when available. Dynamic `info()` data, static definition data, global configuration, and runtime certificates are separate records. `consist()` validates aggregate information only. The catalogue is immutable during a run; UUID/order/definition/weight/brake mismatch reports, stops, and requires recataloguing. Initial catalogue speed is approximately 10 km/h; a higher value requires a bound derived from detector length, minimum spacing, event latency, and `info()` response time.

The Lean boundary is `catalogueProducer` →`StockRecord` →`FrozenProfile` →`sourcePhysicsRecordFromProfile`. Missing info, missing static fields, missing global fields, and definition mismatch return `none`; no neutral physical default is introduced.

6 Directed front-coupler coordinate

Choose the spline orientation permitted by the station/signal marker and let s increase in that direction. If s_{fc} is the front-coupler coordinate, then

$$x = s_{target} - s_{fc}, \qquad v_s = \frac{ds_{fc}}{dt}.$$

The stopping theorem is entered only for $x > 0$ and $v_s > 0$. Wrong-way motion, target crossing, stale data, invalid localization, and unavailable route orientation are recovery or fail-closed states.

If s_c is the tracer-centre coordinate and δ_{fc} is the signed definition/coupler offset in the selected orientation, the localization record is

$$(s_c, \delta_{fc}, s_{fc} = s_c + \delta_{fc}, \epsilon_{fc}).$$

The producer `frontCouplerLocalizationProducer` consumes a validated spline, a complete coupler definition, and a centre error. Raw `getPos()` and a model hitbox are not substitutes. Reverse orientation reverses the directed velocity and selects reverse offsets.

7 IR physics derivation

7.1 Source records and constants

The exact source record contains current mass m, design mass m_d, pitch θ and separately supplied $\sin(\theta)$, slope multiplier M_s, tractive effort T, rolling coefficient c_r, speed-retarder resistance R_s, direct coefficient c_d, interference I, block hardness H, train pressure p_t, independent pressure p_i, brake-system efficiency η_s, adhesion efficiency η_a, global multiplier β, velocity v, shoe coefficient μ_{shoe}, and cogging flag. The Java bytecode evaluates the sine; the rational model requires the runtime adapter to provide it rather than replacing it by zero.

The source constants are

$$\mu_{steel}^{static} = 0.70 = \frac{7}{10}, \quad \mu_{steel}^{kinetic} = 0.42 = \frac{21}{50}, \quad \mu_{steel/castiron}^{kinetic} = 0.25 = \frac{1}{4}, \quad g = 9.8 = \frac{49}{5}.$$

The cast-iron value is recorded as a material-definition fact and source field. The inspected `SimulationState.frictionNewtons()` wheel-slip branch invokes steel/steel kinetic friction, so the proof does not silently substitute 0.25 into that branch. Cogging overrides for brake-system and adhesion efficiency are recorded as bytecode facts, not inferred defaults.

7.2 Line-by-line force ledger

The root bytecode first gives signed grade and traction:

$$F_g = m(-g)\sin(\theta)M_s, \qquad F_T = T.$$

The configuration constructor derives

$$A_d = m_d\,\mu_{steel}^{static}\,g\,\eta_s, \qquad A_{max} = m\,\mu_{steel}^{static}\,g\,\eta_a.$$

The friction method then performs, in order,

$$\begin{aligned}
p &= \min(1, \max(p_t, p_i)),\\
F_{candidate} &= A_d p,\\
F_{roll} &= c_r m g,\\
F_{zero} &= \begin{cases} 0.001mg & v = 0,\\ 0 & v \neq 0, \end{cases}\\
F_{int} &= I\,1000\,H,\\
F_{direct} &= R_s + c_d p_i m g + c_d p_t m g,\\
F_{shoe} &= \begin{cases} m\mu_{steel}^{kinetic} & F_{candidate} > A_{max} \wedge |v| > 0.01,\\ F_{candidate} & \text{otherwise}, \end{cases}\\
F_{brake} &= \beta F_{shoe},\\
F_{net} &= F_g + F_T - (F_{roll} + F_{zero} + F_{direct} + F_{int} + F_{brake}).
\end{aligned}$$

The root wheel-slip branch also sets its sliding state. API command values are validated and normalized at a separate boundary; the root physical pressure branch itself is the upper clamp shown above. This distinction preserves both the source equation and fail-closed command conversion.

The Lean structure `ExactForceLedger` stores every summand and source reference. `source\_force\_ledger\_equation` proves the signed sum and `source\_wheel\_slip\_changes\_branch` proves the slip branch. A missing source field returns `none`; no field is replaced with zero.

7.3 Particle aggregation

For each particle k, the source-shaped transition stores state, interacting mass m_k, interacting friction f_k, previous/next linkage, push/pull capability, slack, collision observation, and track-transition observation. The selected friction is direction-aware and capped by force magnitude; the model update is

$$a_k = \frac{F_k - \text{signedFriction}_k}{m_k}, \quad s_k' = s_k + v_k \Delta t, \quad v_k' = v_k + a_k \Delta t,$$

followed by linkage distance correction. `sourceParticleTransition` rejects invalid mass/linkage and `sourceConsistTransition` preserves per-particle order. This is a conservative source-shaped model; complete Java collision, track lookup, pressure propagation, and all linkage branches remain a refinement obligation.

8 Combined command and plant transition

The command is

$$u = (u_T, u_{train}, u_{ind}, u_E, \alpha_{cmd}, \alpha_{pressure}),$$

with throttle, train brake, independent brake, emergency state, command age, and pressure age. The controller path is explicitly

$$\begin{aligned} &\text{spline localization} \to (x, v_s) \to (m, L, \kappa, \text{slack}, \text{push/pull}) \to u_T \\ &\to (u_{train}, u_{ind}, u_E) \to F_{net} \to \text{particle step} \to \text{next state}. \end{aligned}$$

Positive throttle is not assumed zero after stopping commitment. It is included in the positive-traction upper bound. `apiCommandProducer` rejects nonfinite and out-of-range raw values, `exactCommandPhysics` constructs the command-modified exact ledger, and `exactPlantTransition` applies that ledger to a discrete state transition. `api\_command\_to\_exact\_plant` connects the raw API producer to existence of a next state. Invalid command or linkage input is `none`; this is the fail-closed path.

Saturation and slew are represented as explicit command/runtime fields to be implemented and measured. They may not be omitted because they are inconvenient.

9 Uniform guard, safety, arrival, and holding

Let B_{min} be a lower brake-force bound valid over every declared source branch and profiled particle. It must already account for rolling/direct/ interference/near-zero behavior, wheel slip, material/efficiency branches, and consist aggregation. Let T_{max} bound positive traction, and let $G_{max}, K_{max}, S_{max}, P_{max}$ bound adverse grade, curvature, slack, and push/pull forces. For mass $m > 0$,

$$a_{min} = \frac{B_{min} - T_{max} - G_{max} - K_{max} - S_{max} - P_{max}}{m} > 0.$$

The delay budget is

$$\tau = \tau_{sample} + \tau_{command} + \tau_{pressure} + \tau_{detector} + \tau_{route},$$

where jar-internal next-branch timing is deterministic source evidence and the other terms are measured runtime contracts. For distance d, speed v, localization error e_g, and terminal tolerance e_t, the terminal-entry guard is

$$d > 0, \quad v > 0, \quad a_{min} > 0, \quad d \geq v\tau + \frac{G_{max} + K_{max} + S_{max} + P_{max}}{2m}\tau^2 + \frac{v^2}{2a_{min}} + e_g + e_t.$$

It includes delay distance, adverse delayed motion, stopping distance, geometry error, and terminal tolerance. `terminalGuard` rejects wrong way, crossed target, and nonpositive-deceleration inputs.

The safety envelope is $d \geq 0$, $v \geq 0$, and $v^2 \leq 2a_{min}d$. `arrival\_step\_preserves\_no\_crossing` and `arrival\_run\_velocity\_nonincreasing` prove the explicit rational discrete claims. Terminal arrival is separate: `terminal\_arrival\_step` and `terminal\_arrival\_step\_position` prove $v_s = 0$ and $x = 0$ for the certified terminal step. Runtime acceptance additionally requires

$$|v_s| \leq \epsilon_v, \qquad |x| \leq \epsilon_{stop}, \qquad \text{stable for } N_{stable} \text{ observations.}$$

The position tolerance is derived, not chosen for convenience:

$$\epsilon_{stop} \geq \epsilon_{trace} + \epsilon_{fit} + epsilon_{coupler} + \epsilon_{sample-age} + \epsilon_{command-delay} + \epsilon_{numeric}.$$

Holding uses the actual IR condition $v_s = 0 \wedge |F| < F_{friction}$, not merely zero command or zero force. `holdingTransition` preserves position and zero velocity under that condition. Stable acceptance is a runtime contract, not a certificate that the unchanged Lua already observes the required window.

10 Producer/consumer theorem inventory

Arrow	Producer and consumer	Lean relation and classification
Trace → spline	observations/residuals → validated spline	`validated\_spline\_trace\_fit\_is\_checked`; proved by Lean under trace model
Spline → graph	validated spline/candidates → directed edge/network	`validated\_spline\_to\_directed\_routing\_edge`, `graph\_network\_consumes\_only\_source\_validated\_splines`; proved by Lean
Graph/profile → localization	route/marker/definition → front coupler	`marker\_maps\_to\_valid\_spline\_interval`, `coupler\_localization\_consumes\_validated\_spline`; proved by Lean
Profile → physics	dynamic/static/config/runtime → exact source input	`source\_profile\_physics\_preserves\_mass\_and\_channels`; derived/ proved
Command → plant	numeric/API command → exact ledger/next state	`api\_command\_to\_exact\_plant`, `exact\_plant\_has\_source\_next\_state`; conditional model
Plant → arrival	force/transition → guard, safety, terminal acceptance	`arrival\_step\_preserves\_no\_crossing`, terminal theorems, stable contract; conditional model

10.1 Required spline-to-edge proof

The theorem `validated\_spline\_to\_directed\_routing\_edge` takes the producer equality `validatedSplineProducer c = some v`, orientation o, parameter t, error ϵ, point q, interval premise $0 \leq t \leq 1$, and pointwise premise $\|q - \sigma_c(t)\|_1 \leq \epsilon$. It proves the same stable spline ID, both forward/reverse endpoint correspondences, the directed coordinate mapping, point correspondence through the reverse cubic, source trace IDs, and preservation of the localization inequality. It depends on `traceFitRowsValid`, `splineCandidateValid`, `validatedSplineProducer`, `cubicReverse`, `cubic\_reverse\_position`, `edgeCoordinate`, `directedCoordinateOfParameter`, `edgePointAt`, and `directedEdge`. Its transitive `#print axioms` output is `[propext, Classical.choice, Quot.sound]`. The tests mutate direction, interval, fit row, and spline ID.

11 Anti-cheating audit and gate result

The audit applies the thirteen base anti-cheating classes and the revision's additional checks: no forbidden proof escape; no vacuous producer; no specialized example promoted to a theorem; no equation mismatch; no disconnected definition; no controller-shaped optimality; no sign/dimensional error; no temporal under-approximation; no API/refinement alias; no provenance drift; no rational/floating conflation; no safety/liveness scope confusion; and meaningful negative/mutation tests. Additional audit checks reject optimistic defaults, self-authenticated certificates, and fake transition aliases.

Gates A–H pass only at the explicit proof/contract boundary and are marked conditional where live source or timing evidence is required. Gate I is **NOT_READY**: full Lua source adapters, measured external timing, complete Java linkage/collision refinement, and a live mutation harness remain future obligations. No gate is marked passed merely because Lean or LaTeX compiles.

12 Implementation contract and readiness

The future Lua implementation must persist trace/spline/topology records, derive the graph only from validated splines, map directed markers, catalogue each detector stock immediately, freeze and validate the profile, localize the front coupler, send all command channels together, certify the complete uniform force/delay bound, implement stable terminal holding, and fail closed with diagnostics on every missing or mutated input. It must run the vectors against curved, slack, push/pull, adhesion-limited, wheel-slip, detector-delay, route-mutation, and profile-mutation scenarios.

The final classification is:

- Lean relations and exact vectors: **proved by Lean** under their explicit premises;
- root equations, constants, versions, classes, and branch locations: **derived from bytecode** or bytecode facts;
- trace residual, arc-length approximation, physical force bounds, curvature/slack/push-pull bounds: **certified model assumptions**;
- detector delivery, profile immutability, numeric conversion, timing, route revision, diagnostics, and fail-closed behavior: **runtime contracts**;
- persistent geometry/profile/controller/plant integration and live mutation tests: **future Lua implementation obligations**;
- PID optimality, exact floating-point correctness, complete Java linkage/collision identity, and deployment approval: **unsupported or withdrawn**.

NOT_READY – not approved for user deployment